

网络

1. OSI七层模型和TCP/IP五层模型分别是什么？核心区别是什么？

OSI七层模型是国际标准化组织提出的理论参考模型，从下到上依次为：物理层（比特流传输）、数据链路层（帧传输、MAC地址）、网络层（路由、IP地址）、传输层（端到端连接、TCP/UDP）、会话层（建立管理会话）、表示层（数据格式转换、加密压缩）、应用层（HTTP、FTP等）。TCP/IP五层模型是实际使用的工业标准，它将OSI的会话层、表示层、应用层合并为一层“应用层”，因此成为：物理层、数据链路层、网络层、传输层、应用层。核心区别：OSI更严谨、分层更细，但实现复杂；TCP/IP更简洁、实用，是互联网的基础。面试时可以说：OSI是教学模型，TCP/IP是实际部署模型。

2. 什么是IP地址？IPv4和IPv6的区别是什么？

IP地址是互联网协议地址，用于在网络中唯一标识一台主机（或网络接口），属于网络层。IPv4：32位二进制，通常用点分十进制表示（如192.168.1.1），理论地址总数约42亿，目前已枯竭。IPv6：128位二进制，用冒号分隔的十六进制表示（如2001:0db8:85a3::8a2e:0370:7334），地址数量极多（ 2^{128} ），解决了IPv4短缺问题，并且内置了IPSec安全机制、简化了报文头部、支持自动配置等。面试时补充：IPv6中取消了广播，使用组播和任播。

3. 什么是子网掩码？作用是什么？

子网掩码是一个32位二进制数，与IP地址成对使用。其中值为1的位对应IP地址的网络部分，值为0的位对应主机部分。作用有三：① 区分IP地址的网络号和主机号；② 划分子网，将一个大的网络分成多个小网络，减少广播域；③ 判断两个IP是否属于同一子网：将IP和子网掩码做按位与运算，若结果相同则在同一子网。例如：IP 192.168.1.10，子网掩码255.255.255.0，则网络地址为192.168.1.0，主机地址为0.0.0.10。

4. TCP和UDP的区别？

TCP是面向连接的、可靠的、基于字节流的传输层协议；UDP是无连接的、不可靠的、基于数据报的协议。具体区别如下：连接性：TCP需要三次握手建立连接，UDP不需要。可靠性：TCP提供确认应答、超时重传、序列号、校验和等机制保证数据完整有序；UDP只提供校验和，不保证不丢失、不重复、不乱序。效率：TCP首部最小20字节，UDP首部仅8字节；TCP有拥塞控制和流量控制，可能降低发送速度，UDP则没有，发送更快。应用场景：TCP用于文件传输、邮件、网页（HTTP）等要求可靠的场景；UDP用于实时音视频、DNS查询、SNMP等允许丢包但要求低延迟的场景。

5. TCP三次握手的过程是怎样的？

① 第一次握手：客户端向服务器发送一个SYN报文段 ($seq=x$)，表示请求建立连接，此时客户端进入SYN_SENT状态。② 第二次握手：服务器收到SYN后，回复一个SYN+ACK报文段 ($seq=y, ack=x+1$)，表示同意建立连接，并确认收到客户端的SYN，服务器进入SYN_RCVD状态。③ 第三次握手：客户端收到SYN+ACK后，再发送一个ACK报文段 ($seq=x+1, ack=y+1$)，表示确认服务器的SYN，之后双方进入ESTABLISHED状态，连接建立完成。面试扩展：为什么是三次？因为两次无法确认客户端的接收能力正常，且三次可以避免历史连接的重复初始化。

6. TCP四次挥手的过程是怎样的？

① 第一次挥手：主动关闭方（比如客户端）发送一个FIN报文段 ($seq=u$)，表示不再发送数据，进入FIN_WAIT_1状态。② 第二次挥手：被动关闭方收到FIN后，回复ACK ($ack=u+1$)，表示已收到关闭请求，但可能还有数据要发，进入CLOSE_WAIT状态。主动方收到ACK后进入FIN_WAIT_2状态。③ 第三次挥手：被动方数据发送完毕后，发送FIN ($seq=v, ack=u+1$)，进入LAST_ACK状态。④ 第四次挥手：主动方收到FIN后，回复ACK ($ack=v+1$)，进入TIME_WAIT状态，等待2MSL（最大报文生存时间）后关闭；被动方收到ACK后立即关闭。追问：为什么需要TIME_WAIT？为了确保最后一个ACK能被对方收到，并让网络中残留的旧报文段消失。

7. 为什么TCP握手是3次，挥手是4次？

握手3次：服务器在收到SYN后，可以将SYN和ACK合并为一个报文发送，因为此时服务器没有需要等待的数据发送，所以三次足够。挥手4次：当主动方发送FIN时，仅表示主动方不再发送数据，但被动方可能还有未发送完的数据，因此被动方的ACK和FIN不能合并，需要先回复ACK表示收到关闭请求，等数据发完后再发FIN，所以需要四次。

8. TCP如何保证可靠传输？

TCP通过多种机制保证可靠传输：① 校验和：对首部和数据进行校验，检测损坏的报文段。② 序列号和确认应答：每个字节都有序列号，接收方回复ACK表示已收到，发送方根据ACK知道哪些数据被确认。③ 超时重传：发送方启动定时器，若未在RTO（重传超时时间）内收到ACK则重传该报文段。④ 流量控制：通过滑动窗口，接收方通告窗口大小，防止发送方发送过快导致接收方缓冲区溢出。⑤ 拥塞控制：包括慢启动、拥塞避免、快重传、快恢复，避免网络过载。这些机制共同保证了数据的无差错、不丢失、不重复、按序到达。

9. TCP的拥塞控制算法有哪些？

TCP拥塞控制主要包含四个算法：① 慢启动：连接建立后，拥塞窗口

(cwnd) 从1个MSS开始，每收到一个ACK窗口加倍，指数增长直到达到慢启动阈值 (sssthresh)。② 拥塞避免：当cwnd $\geq$ sssthresh时，每个RTT窗口增加1个MSS (线性增长)，直到出现丢包。③ 快重传：当发送方收到三个重复ACK时，认为丢包，立即重传丢失的报文段，而不必等待超时。④ 快恢复：快重传后，将sssthresh设为当前cwnd的一半，cwnd设为sssthresh+3，然后进入拥塞避免阶段。(早期TCP Tahoe没有快恢复，会直接回到慢启动。)

10. TCP滑动窗口的作用？

滑动窗口有两个作用：① 流量控制：接收方通过TCP首部中的“窗口”字段告诉发送方自己还能接收多少字节 (即接收缓冲区剩余大小)，发送方据此调整发送速率，避免接收方被淹没。② 提高吞吐量：窗口大小决定了在收到确认之前可以连续发送的数据量，窗口越大，网络利用率越高。滑动窗口使得发送方不必停等每个ACK，可以连续发送窗口内的数据。滑动窗口由发送窗口和接收窗口组成，窗口可以向左滑动 (确认数据被接收)，也可以向右扩大或缩小 (由接收方通告)。

11. HTTP和HTTPS的区别？

① 安全性：HTTP是明文传输，数据可被窃听或篡改；HTTPS使用SSL/TLS协议对通信进行加密，提供机密性、完整性和身份认证。② 端口：HTTP默认80端口，HTTPS默认443端口。③ 证书：HTTPS需要向CA (证书颁发机构) 申请数字证书，证明服务器身份；HTTP不需要。④ 性能：HTTPS因加解密和证书验证过程，比HTTP慢一些，但现代硬件和优化协议已使差距很小。⑤ URL前缀：HTTP为http://，HTTPS为https://。补充：HTTPS并非完全安全，如果证书被伪造或用户忽略证书警告，仍可能被中间人攻击。

12. HTTPS的加密过程是怎样的？

HTTPS采用混合加密机制：① 客户端发起ClientHello，包含支持的加密套件、TLS版本、随机数 (Client Random)。② 服务器回复ServerHello，选定加密套件、服务器随机数 (Server Random)，并发送数字证书 (包含服务器公钥)。③ 客户端验证证书 (检查是否过期、域名是否匹配、是否由信任的CA签发等)，若通过则生成一个随机数 (Pre-Master Secret)，用服务器公钥加密后发送给服务器。④ 服务器用私钥解密得到Pre-Master Secret，双方根据Client Random、Server Random、Pre-Master Secret计算出会话密钥 (对称密钥)。⑤ 之后所有通信都用该会话密钥进行对称加密。追问：为什么非对称加密只用来交换密钥？因为非对称加密慢，对称加密快，所以组合使用。

13. HTTP有哪些常见状态码？

1xx (信息性)：100 Continue，客户端可以继续发送请求。2xx (成功)：200 OK，请求成功；201 Created，资源创建成功 (常用于POST)。3xx (重定

向)：301 Moved Permanently，资源永久移动到新URL；302 Found，临时移动；304 Not Modified，缓存未修改，可继续使用缓存。4xx（客户端错误）：400 Bad Request，请求语法错误；401 Unauthorized，需要身份认证；403 Forbidden，服务器拒绝；404 Not Found，资源不存在。5xx（服务端错误）：500 Internal Server Error，服务器内部错误；502 Bad Gateway，网关错误；503 Service Unavailable，服务器超载或停机维护。记忆技巧：1xx消息、2xx成功、3xx重定向、4xx客户端错、5xx服务端错。

14. GET和POST的区别？

① 语义：GET用于获取资源，POST用于提交数据（如创建或修改资源）。② 参数位置：GET参数放在URL的查询字符串中，POST参数放在请求体中（也可以放URL，但通常不放）。③ 长度限制：GET受浏览器/服务器对URL长度限制（一般2KB~8KB），POST无限制（受服务器配置限制）。④ 安全性：GET参数暴露在URL中，易被浏览器历史、日志记录，不适合敏感信息；POST相对安全，但仍需HTTPS加密。⑤ 幂等性：GET是幂等的，多次请求结果相同；POST非幂等，多次请求可能创建多个资源。⑥ 缓存：GET请求可以被浏览器缓存，POST默认不缓存。注意：上述区别是HTTP规范约定，实际实现中不一定严格遵守。

15. Session和Cookie的区别？

存储位置：Cookie存储在客户端（浏览器），Session存储在服务器端（内存或数据库）。大小限制：Cookie一般不超过4KB，Session无大小限制（但过多会消耗服务器内存）。安全性：Cookie可能被篡改或窃取，Session相对安全，但Session ID若通过Cookie传输，仍需防范劫持。生命周期：Cookie可以设置过期时间，持久保存；Session默认随会话结束（关闭浏览器或超时）失效。跨域：Cookie默认同源访问，可设置Domain实现跨子域；Session依赖于Session ID的传递机制，通常通过Cookie，也可通过URL重写。典型用法：用户登录后，服务器创建Session并返回Session ID（存入Cookie），后续请求携带Cookie，服务器根据Session ID识别用户。

16. 在浏览器输入URL后发生了什么？

这是一个综合性问题，分步骤回答：① DNS解析：浏览器检查缓存、hosts文件，向本地DNS服务器发起查询，递归/迭代得到域名对应的IP地址。② TCP连接：浏览器与服务器建立TCP连接（三次握手）。③ 发送HTTP请求：浏览器构造HTTP请求报文（请求行、头部、空行、主体），通过TCP发送。④ 服务器处理并响应：服务器接收请求，处理（如查询数据库、执行代码），返回HTTP响应（状态码、头部、响应体）。⑤ 浏览器解析渲染：浏览器解析HTML，构建DOM树；解析CSS生成CSSOM树；合成渲染树；布局绘制；执行JS。⑥ 资源加载：解析过程中遇到图片、CSS、JS等资源，重复上述过程

(可能并发多个请求)。⑦ 连接关闭：若Connection为close，则TCP四次挥手关闭；若为keep-alive，则保持连接以复用。

17. DNS解析过程是怎样的？

以访问www.example.com为例：① 浏览器检查自身DNS缓存，若有则直接返回。② 若无，检查操作系统hosts文件。③ 若没有，向本地DNS服务器（通常是ISP提供的，或手动配置的如114.114.114.114）发起递归查询请求。④ 本地DNS服务器若没有缓存，则从根域名服务器开始迭代查询：根域返回.com顶级域的NS记录和地址；本地DNS再向.com顶级域查询，返回example.com的权威域名服务器地址；本地DNS再向example.com的权威服务器查询，得到www.example.com的IP地址。⑤ 本地DNS将结果返回给客户端，并缓存该记录。补充：DNS使用UDP 53端口，若响应超过512字节则使用TCP。

18. ARP协议是什么？它的作用是什么？

ARP（地址解析协议）用于将网络层的IP地址解析为数据链路层的MAC地址。在同一局域网内，发送方知道目标IP但不知道其MAC地址时，会广播一个ARP请求包，包含目标IP；目标主机收到后单播回复自己的MAC地址。发送方将IP-MAC映射存入ARP缓存，后续通信直接使用。典型场景：主机A（192.168.1.2）想与主机B（192.168.1.3）通信，A广播ARP请求“谁有192.168.1.3？”，B回复“我的MAC是xx:xx:xx:xx:xx:xx”，A即可封装以太网帧发送。安全注意：ARP无认证机制，易受ARP欺骗攻击。

19. Cookie和Session的区别？

（与第15题相同，可参照）

20. DoS和DDoS攻击分别是什么？

DoS（拒绝服务攻击）：单一攻击源向目标发送大量请求或利用协议漏洞，消耗目标资源（带宽、CPU、内存），使其无法正常服务合法用户。例如SYN Flood攻击，发送大量SYN但不回复ACK，耗尽服务器连接队列。DDoS（分布式拒绝服务攻击）：攻击者控制大量僵尸主机（肉鸡）同时向目标发起流量攻击，规模更大，更难防御。常见类型：流量型（UDP Flood、ICMP Flood，占满带宽）、连接型（SYN Flood、HTTP Flood，耗尽连接或应用资源）、慢速攻击（Slowloris，缓慢发送HTTP请求占用连接）。防御手段：流量清洗、限流、CDN、高防IP、验证码等。

数据库

1. InnoDB和MyISAM的核心区别是什么？

事务支持：InnoDB支持ACID事务，MyISAM不支持。锁粒度：InnoDB默认行级锁（通过索引实现），并发写入性能好；MyISAM表级锁，写操作会锁全表，不适合高并发写。外键：InnoDB支持外键约束，MyISAM不支持。崩溃恢复：InnoDB通过redo log和undo log实现崩溃恢复，MyISAM崩溃后易损坏，需修复。存储结构：InnoDB数据和索引集中存储（聚簇索引），MyISAM数据、索引、压缩表分开存储。适用场景：InnoDB适合OLTP（高并发读写、需要事务）；MyISAM适合OLAP（只读、批量插入、全文索引）。MySQL 5.5以后InnoDB为默认引擎。

2. 事务的四大特性（ACID）是什么？

原子性：事务中的所有操作要么全部成功提交，要么全部失败回滚，不会停留在中间状态。通过Undo Log实现：记录操作前的数据，回滚时恢复。一致性：事务执行前后，数据库的完整性约束没有被破坏（如主键、外键、check约束、业务规则）。一致性由原子性、隔离性、持久性共同保证。隔离性：多个事务并发执行时，彼此之间不可见，如同串行执行。通过锁机制和MVCC实现。持久性：事务一旦提交，其对数据库的修改是永久性的，即使系统崩溃也不会丢失。通过Redo Log实现：提交前先写redo log，崩溃后重做。

3. MySQL的隔离级别有哪些？

SQL标准定义了四种隔离级别，从低到高：读未提交（Read Uncommitted）：事务可以读取其他事务未提交的数据（脏读）。读已提交（Read Committed）：只能读取已提交的数据，解决脏读，但可能出现不可重复读（同一事务内两次读取同一行结果不同）。可重复读（Repeatable Read）：同一事务内多次读取同一数据结果一致，解决不可重复读，但可能出现幻读（事务内两次范围查询结果集不同）。MySQL InnoDB通过MVCC+间隙锁解决了幻读。串行化（Serializable）：强制事务串行执行，解决所有并发问题，但性能最差。MySQL默认隔离级别是可重复读。不同级别的并发问题如下表：读未提交可能出现脏读、不可重复读、幻读；读已提交不可能脏读，可能不可重复读和幻读；可重复读不可能脏读和不可重复读，但可能幻读（InnoDB解决）；串行化都不可能出现。

4. MVCC（多版本并发控制）的实现原理？

MVCC使得InnoDB在可重复读隔离级别下，实现“读不加锁，读写不冲突”。其核心组件：隐藏列：每行记录包含三个隐藏字段：DB_TRX_ID（最后修改的事务ID）、DB_ROLL_PTR（指向undo log的指针）、DB_ROW_ID（单调递增的行ID，若没有主键则自动生成聚簇索引）。Undo Log：记录数据的历史版本。当更新一行时，旧版本被写入undo log，新版本的新DB_TRX_ID为当前事务ID，DB_ROLL_PTR指向旧版本。不同事务通过指针形成版本链。Read

View：在事务执行快照读时生成，包含以下信息：creator_trx_id（当前事务ID）、up_limit_id（未提交的最小事务ID）、low_limit_id（下一个要分配的事务ID）、trx_ids（当前活跃的读写事务ID列表）。可见性判断：遍历版本链，如果某版本的DB_TRX_ID满足以下条件之一则可见：等于当前事务ID（自己修改的）；小于up_limit_id（已提交）；在trx_ids中不存在且小于low_limit_id（已提交）。否则继续回滚指针找更旧版本。快照读（普通SELECT）使用MVCC，当前读（SELECT ... FOR UPDATE/LOCK IN SHARE MODE）使用锁。

5. MySQL为什么选择B+树作为索引结构？

磁盘I/O友好：B+树的非叶子节点只存键值和指针，不存数据，一个磁盘页可以存储更多索引条目，树的高度更低（通常3~4层），减少磁盘I/O次数。范围查询高效：B+树所有叶子节点通过双向链表相连，进行范围查询（如WHERE id BETWEEN 10 AND 20）时，只需找到起点，然后沿着链表顺序遍历即可，而B树则需要多次回溯父节点。查询稳定性：所有查询都要走到叶子节点，时间复杂度稳定为 $O(\log n)$ ，而B树的查询可能提前在非叶子节点命中，但不稳定。数据存储集中：叶子节点存储完整的数据行（聚簇索引）或主键值（二级索引），顺序I/O更高效。对比哈希索引：哈希适合等值查询，但不支持范围查询和排序。

6. B+树和哈希索引的区别？

数据结构：B+树是平衡多叉树，哈希索引是哈希表。查询类型：B+树支持等值、范围、前缀、排序；哈希索引仅等值（=、IN）。等值查询效率：B+树为 $O(\log n)$ ，哈希索引理想 $O(1)$ 但有哈希碰撞时退化为链表。范围查询：B+树支持，哈希索引不支持。排序：B+树支持ORDER BY无需额外排序，哈希索引不支持，返回无序。索引列运算：两者都对索引列使用函数会导致失效。使用引擎：InnoDB、MyISAM默认B+树，哈希索引仅Memory引擎支持，且关闭自适应哈希时不用。

7. 什么是索引的最左前缀原则？

最左前缀原则是指：联合索引（a, b, c）会先按a排序，再按b，再按c。查询时必须从索引的最左列开始匹配，不能跳过中间的列。有效情况：WHERE a = 1 走索引；WHERE a = 1 AND b = 2 走索引；WHERE a = 1 AND b = 2 AND c = 3 走索引；WHERE a = 1 AND c = 3 中a走索引，c无法使用，因为b被跳过；WHERE b = 2 没有a，无法使用该索引。注意：MySQL优化器会调整条件顺序，但不会改变对最左列的要求。应用：建立联合索引时，应将区分度高、查询频率高的列放在左边。

8. Explain的关键字段如何解读？

使用EXPLAIN SELECT ...查看执行计划，关键字段：id：查询的序列号，越大越先执行；相同则从上到下执行。select_type：查询类型，如SIMPLE（简单

查询)、PRIMARY (最外层)、SUBQUERY (子查询)、DERIVED (派生表)。
table: 正在访问的表。type: 访问类型, 从好到坏依次: system > const > eq_ref > ref > range > index > ALL。const: 主键或唯一索引等值查询, 最多匹配一行。ref: 非唯一索引等值查询。range: 范围查询 (BETWEEN、>、<)。index: 扫描整个索引树 (比ALL好, 因为比数据表小)。ALL: 全表扫描, 最差。possible_keys: 可能使用的索引。key: 实际使用的索引。
key_len: 索引使用的字节数, 可用于判断复合索引使用了多少列。rows: 预估扫描的行数, 越小越好。Extra: 额外信息, 如Using index (覆盖索引)、Using where、Using filesort (需要额外排序)、Using temporary (使用临时表)。

9. 索引有哪些失效情况?

对索引列使用函数或表达式: 如WHERE YEAR(date) = 2020, 应改为WHERE date BETWEEN '2020-01-01' AND '2020-12-31'。隐式类型转换: 如WHERE phone = 13800000000, phone字段是varchar, 但传入整数, MySQL会转换类型导致索引失效。LIKE以通配符开头: WHERE name LIKE '%abc'无法使用索引, WHERE name LIKE 'abc%'可以。OR条件中有非索引列: 如WHERE a = 1 OR b = 2, 若b无索引, 则整个查询不走索引。联合索引不满足最左前缀: 如索引(a,b,c), WHERE b = 1失效。数据分布不均: 若MySQL认为全表扫描更快 (如查询结果超过表30%), 会放弃索引。使用!=或<>: 通常导致索引失效 (除非覆盖索引)。IS NULL / IS NOT NULL: 某些情况下可能失效, 取决于NULL值比例。

10. 什么是聚簇索引? 什么是非聚簇索引?

聚簇索引: 索引的叶子节点存储的是完整的行数据。InnoDB中, 主键就是聚簇索引。如果没有显式定义主键, 会选择第一个唯一非空索引作为聚簇索引; 都没有则自动生成一个隐藏的row_id。聚簇索引决定了数据行的物理存储顺序, 一个表只能有一个聚簇索引。非聚簇索引 (二级索引): 索引的叶子节点存储的是主键值 (InnoDB) 或指向数据行的指针 (MyISAM)。查询时先找到主键值, 再通过主键到聚簇索引中回表获取完整行。回表: 使用二级索引查询时, 先查到主键, 再到聚簇索引查一次数据。为了减少回表, 可以使用覆盖索引 (索引中包含了查询所需的所有字段)。

11. 乐观锁和悲观锁的区别?

悲观锁: 假设并发冲突一定会发生, 每次操作数据时都加锁, 阻塞其他事务直到自己释放锁。实现方式: SELECT ... FOR UPDATE (行级锁)、表锁等。适用写多读少、冲突严重的场景。乐观锁: 假设并发冲突很少发生, 操作数据时不加锁, 只在提交更新时检查是否被其他事务修改。常用实现: 版本号 (version) 或时间戳。SQL示例: UPDATE table SET count = count - 1,

`version = version + 1 WHERE id = 1 AND version = old_version;` 如果影响行数为0，说明版本号已变，则重试或抛异常。适用读多写少、冲突低的场景。对比：悲观锁保证强一致性但降低并发；乐观锁性能好但需要重试机制。

12. 什么是慢查询？如何优化？

慢查询指执行时间超过指定阈值（如1秒）的SQL语句。定位：开启MySQL慢查询日志（`slow_query_log=ON`，设置`long_query_time`），分析日志找到慢SQL。优化步骤：① 使用EXPLAIN分析执行计划，重点关注type、rows、Extra。② 根据分析结果优化：添加或调整索引（覆盖索引、联合索引顺序）；避免SELECT *，只取需要的字段；优化JOIN：小表驱动大表，确保关联字段有索引；拆分复杂查询为多个简单查询，或用临时表；避免子查询，改写成JOIN（某些版本子查询性能差）；分批处理大范围更新/删除。③ 考虑数据库结构优化：分库分表、归档历史数据、调整buffer pool等。工具：pt-query-digest分析慢日志，performance_schema查看等待事件。

13. 主从同步的原理是什么？

MySQL主从复制基于binlog实现，异步或半同步复制。流程：① 主库：事务提交时，将数据变更写入二进制日志（binlog）。② 从库：启动两个线程——I/O线程：连接到主库，请求从指定位置开始读取binlog，主库发送binlog事件，I/O线程接收并写入从库的中继日志（relay log）。SQL线程：读取中继日志，解析并重放SQL语句，保持与主库数据一致。复制类型：异步复制（主库提交后不管从库，性能高但可能丢数据）、半同步复制（主库至少等待一个从库确认收到binlog后才返回成功）、全同步复制（所有从库确认后才返回，几乎不用）。延迟问题：从库SQL线程单线程回放慢，MySQL 5.7引入多线程并行复制（基于库或逻辑时钟）。

14. 如何解决“库存超卖”问题？

超卖指在高并发秒杀场景下，库存被扣减超过实际数量。常用方案：① 数据库乐观锁：`UPDATE product SET stock = stock - 1 WHERE id = 1 AND stock > 0;` 检查影响行数，若为0则重试或失败。② 数据库悲观锁：`SELECT stock FROM product WHERE id = 1 FOR UPDATE;` 业务判断后更新 `UPDATE product SET stock = stock - 1 WHERE id = 1;` 注意事务要尽快提交，避免长事务。③ Redis原子操作：使用Redis的DECR或Lua脚本保证原子性，再异步同步到数据库。④ 消息队列：将所有请求放入队列，单消费者串行处理，避免并发。⑤ 分布式锁：如Redisson，但性能较差。推荐：Redis+Lua + 数据库乐观锁相结合，兼顾性能与一致性。

15. 什么是数据库的三大范式？

范式是数据库设计规范，减少数据冗余和更新异常。第一范式（1NF）：每个列都是不可分割的原子数据项。例如“联系方式”列不能包含电话和邮箱两个

值，需拆成两列。第二范式（2NF）：在1NF基础上，非主键列必须完全依赖主键（不能部分依赖）。适用于联合主键，例如订单明细表（订单ID，产品ID，产品名称，数量），产品名称只依赖产品ID（部分依赖），应拆分为产品表和订单明细表。第三范式（3NF）：在2NF基础上，非主键列不能传递依赖于主键。例如员工表（员工ID，部门ID，部门名称），部门名称依赖于部门ID，而部门ID依赖于员工ID，存在传递依赖，应拆分为部门表。注意：实际开发中常为了性能而反范式化，适当冗余数据减少JOIN。

16. SQL注入是什么？如何防范？

SQL注入是攻击者通过构造恶意输入，使应用程序拼接成的SQL语句被改变语义，从而执行非预期的数据库操作。示例：登录查询SELECT * FROM user WHERE name = 'name' AND pwd = 'pwd'，攻击者输入name = "admin' --"，SQL变为SELECT * FROM user WHERE name = 'admin' --' AND pwd = 'xxx'，注释掉密码验证。防范措施：① 参数化查询（PreparedStatement）：将SQL语句与参数分开，参数仅作为数据处理，不会被解释为SQL指令。这是最有效的方法。② 输入校验与过滤：对输入进行白名单校验（如数字、枚举值），或转义特殊字符（如'变为'），但不如参数化查询安全。③ 最小权限原则：数据库账号仅授予必要权限（如只读用户不能执行DROP）。④ 使用ORM框架：如MyBatis的#{ }是参数化，\${ }仍有风险，应避免。

17. 内连接、左连接、右连接的区别？

INNER JOIN：返回两个表中连接字段匹配的行。若某行在另一表中无匹配，则不返回。LEFT JOIN：返回左表所有行，右表匹配的行有数据则显示，无匹配则填充NULL。RIGHT JOIN：返回右表所有行，左表匹配的行有数据则显示，无匹配则填充NULL。举例：学生表（id, name），选课表（student_id, course）。内连接：只返回有选课记录的学生。左连接：返回所有学生，没有选课的学生课程字段为NULL。右连接：返回所有选课记录，但若选课表中的student_id不存在于学生表，则学生信息为NULL（通常不会发生，因为应有外键）。注意：MySQL不支持FULL OUTER JOIN，可用LEFT JOIN UNION RIGHT JOIN模拟。

18. CHAR和VARCHAR的区别？

CHAR(n)：定长字符串，最大255字符。存储时总是占用n个字节（取决于字符集，如UTF8下n个字符可能占3n字节）。检索时会去掉尾部空格。适合存储长度固定的数据，如手机号、身份证号、MD5值。VARCHAR(n)：变长字符串，最大65535字节（实际受行大小限制）。存储时额外占用1~2字节记录长度。不填充空格，节省空间。适合长度变化大的数据，如评论、地址。性能：CHAR在更新时不易产生页分裂，但可能浪费空间；VARCHAR节省空间但可能引起碎片。选择建议：短且长度固定用CHAR，否则用VARCHAR。

19. 为什么要分库分表？

当单库或单表数据量过大（如MySQL单表超过千万行），或并发请求过高（QPS过万）时，传统优化手段（索引、缓存、SQL优化）已不够，需分库分表。垂直分表：将宽表按字段拆分为多个表，如把热点字段和冷字段分开，减少单表宽度，提高缓存命中率。垂直分库：按业务模块拆分数数据库，如用户库、订单库，分散到不同服务器。水平分库分表：将同一张表的数据按某种规则（如hash、range）分布到多个库或表中。水平分表解决单表数据量过大，查询扫描行数减少；水平分库解决单库连接数、磁盘IO、CPU压力。常见中间件：ShardingSphere、MyCat、Vitess。挑战：跨库JOIN、分布式事务、全局主键、分页排序等需特殊处理。

20. Redis缓存穿透、缓存雪崩、缓存击穿是什么？

缓存穿透：查询一个不存在的数据，由于缓存中没有，请求会直接打到数据库，若恶意攻击大量不存在key，会导致数据库压力过大。解决：布隆过滤器（先过滤一定不存在的key）；缓存空对象（设置较短过期时间）。缓存雪崩：大量缓存同时过期，或者Redis宕机，导致请求全部打到数据库，数据库可能崩溃。解决：过期时间加上随机值；热点数据永不过期（异步更新）；Redis高可用（集群、哨兵）；加锁或限流。缓存击穿：一个热点key在过期瞬间，高并发请求同时发现缓存失效，同时查询数据库，导致数据库压力骤增。解决：互斥锁（SETNX），只有一个线程查数据库并重建缓存；永不过期+后台异步更新。

Linux

1. Linux的体系结构是怎样的？

Linux体系分为用户空间和内核空间，通过系统调用接口交互。内核空间：包括进程调度、内存管理、文件系统、网络协议栈、设备驱动等子系统。内核运行在最高特权级（Ring 0），可以直接访问硬件。用户空间：运行用户应用程序（如Shell、Web服务器）和C库（glibc），通过系统调用（如open、read、write）请求内核服务。特点：内核提供抽象层，使应用程序无需关心硬件差异；模块化设计，支持动态加载内核模块。

2. 绝对路径、相对路径、当前目录、上层目录、主目录如何表示？

绝对路径：从根目录/开始写起的完整路径，如/home/user/file.txt。相对路径：相对于当前工作目录的路径，不以/开头，如docs/file.txt。当前目录：用.表示。上层目录：用..表示。主目录：用~表示，对于root用户是/root，普通用户是/home/用户名。切换目录：cd /path。查看当前目录：pwd（print working directory）。

3. 如何查看当前进程？如何退出？如何查看当前路径？

查看进程：ps aux：显示所有用户的所有进程，包含详细状态（CPU、内存等）。ps -ef：显示全格式的进程列表。top：动态实时显示进程，按q退出。退出：在终端中，exit命令退出当前shell或用户；在top等交互界面，按q退出。查看当前路径：pwd命令。

4. 如何查看内核版本？

使用uname -r命令。uname用于打印系统信息，常用参数：-a：显示所有信息（内核名、主机名、内核版本、硬件架构等）；-r：仅显示内核版本号，例如5.4.0-26-generic。另外，cat /proc/version也可以查看内核版本及gcc版本。

5. 硬链接和软链接的区别？

硬链接：ln source target。多个文件名指向同一个inode（索引节点），删除一个文件名不影响其他文件名访问数据。硬链接不能跨文件系统，不能链接目录（防止循环）。软链接（符号链接）：ln -s source target。相当于快捷方式，文件内容存储源文件的路径。软链接可以跨文件系统，可以链接目录；删除源文件后，软链接变成“悬空链接”。查看方式：ls -li显示inode号，硬链接的inode号相同，软链接的文件类型为l。

6. 如何修改文件权限？格式是怎样的？

使用chmod命令修改权限。权限分为属主（u）、属组（g）、其他用户（o）。数字法：每个权限位用数字表示：r=4，w=2，x=1。例如chmod 754 file：7=rwx（属主），5=r-x（属组），4=r--（其他）。字母法：chmod u+rwx,g+rx,o+x file；也可用=设置精确权限，-移除权限。递归修改目录：chmod -R 755 dir。

7. 有哪些查看文件内容的命令？

cat：一次性显示整个文件内容，适合小文件。more：分页显示，按空格翻页，按q退出。less：功能更强大，可以上下翻页（PgUp/PgDn）、搜索（/关键字），按q退出。head -n 20：显示文件前20行。tail -n 20：显示文件后20行；tail -f实时追踪文件新增内容，常用于查看日志。nl：显示行号并输出内容。

8. 复制、移动、删除文件的命令是什么？

复制：cp source dest。复制目录加-r（递归）。常用-i（覆盖前询问），-p（保留属性），-a（归档，等于-dR --preserve=all）。移动/改名：mv source dest。如果目标在同一分区，只是修改文件名；否则会复制并删除源文件。删除：rm file。删除目录加-r；强制删除加-f（不提示）；rm -rf dir非常危险，慎用。安全建议：避免使用rm -rf /。

9. 创建目录和创建文件的命令是什么？

创建目录：mkdir dirname。创建多级目录用mkdir -p /a/b/c。创建文件：touch filename：创建空文件或更新文件时间戳。vi filename：用vim编辑器创建并编辑。echo "content" > filename：重定向创建文件。cat > filename：然后输入内容，Ctrl+D结束。

10. 如何查看系统进程？动态查看用什么命令？

静态查看：ps aux（BSD风格，显示USER、PID、%CPU、%MEM、VSZ、RSS、TTY、STAT、START、TIME、COMMAND）。ps -ef（System V风格，显示UID、PID、PPID、C、STIME、TTY、TIME、CMD）。动态查看：top命令，实时刷新进程列表，按P按CPU排序，按M按内存排序，按H显示线程，按q退出。其他：htop（更友好的交互界面，可能需要安装），pidstat（查看特定进程的CPU、内存、IO）。

11. Linux有哪些常见的进程间通信方式？

Linux进程间通信（IPC）方式包括：① 管道：匿名管道（|）用于父子进程；命名管道（FIFO）用于无亲缘关系进程。② 信号：如SIGKILL、SIGTERM，用于异步事件通知。③ 消息队列：消息的链表，存储在内存中，消息有类型，可按类型读取。④ 共享内存：最快的IPC，多个进程映射同一块物理内存，但需同步机制（如信号量）。⑤ 信号量：用于进程间同步，控制对共享资源的访问（类似锁）。⑥ Socket：可用于不同主机间的进程通信，也可用于本机（Unix domain socket）。⑦ 内存映射文件（mmap）：将文件映射到进程地址空间，实现共享。面试时可根据场景重点介绍共享内存+信号量组合。

12. 如何终止进程？

使用kill命令向进程发送信号，默认发送SIGTERM（15），允许进程清理后退出。kill PID：发送SIGTERM，进程可捕获并优雅退出。kill -9 PID：发送SIGKILL，强制立即终止，进程无法捕获。kill -l：列出所有信号名称。其他命令：pkill（按进程名杀），killall（杀所有匹配的进程）。注意：强制终止可能导致数据丢失或资源未释放。

13. 如何查看CPU和内存使用情况？

CPU：top或htop：实时显示各进程CPU占用率及总CPU使用率。mpstat -P ALL 1：显示每个CPU核心的使用率，每秒刷新。vmstat 1：显示系统整体CPU、内存、swap、IO情况。内存：free -h：以人类可读格式显示总内存、已用、空闲、buff/cache、可用。cat /proc/meminfo：更详细的内存信息。top中按M按内存排序。综合：sar（系统活动报告工具，需安装sysstat）。

14. Linux有哪些常见的压缩包格式？如何压缩？

常见格式及对应命令：tar.gz：tar -czvf archive.tar.gz /path（c创建，z通过

gzip压缩, v显示过程, f指定文件名)。tar.bz2: tar -cjvf archive.tar.bz2 /path (j表示bzip2)。tar.xz: tar -cjvf archive.tar.xz /path (J表示xz)。zip: zip -r archive.zip /path (-r递归目录)。7z: 7z a archive.7z /path (需安装p7zip)。解压: tar -xzvf archive.tar.gz (x解压); unzip archive.zip; 7z x archive.7z。

15. 如何查看后台任务? 如何将后台任务调到前台?

查看后台任务: jobs命令列出当前shell中在后台运行或暂停的任务, 会显示任务编号 (如[1])、状态 (Running/Stopped)、命令。将后台任务调到前台: fg %任务编号, 例如fg %1。如果没有编号, fg默认调回最近的后台任务。将任务放到后台运行: 命令后加&: command &, 直接后台运行。运行中的任务按Ctrl+Z暂停, 然后输入bg %任务编号使其在后台继续运行。注意: 关闭终端后后台任务会收到SIGHUP而退出, 可使用nohup或disown避免。

16. 如何查看端口是否被占用?

常用三种命令: ① netstat -tunlp | grep 端口号 (-t: TCP, -u: UDP, -n: 不解析域名, -l: 监听状态, -p: 显示进程PID/名称)。② lsof -i:端口号, 例如lsof -i:80, 显示占用80端口的进程。③ ss -tunlp | grep 端口号 (ss比netstat更快, 用法类似)。示例: netstat -tunlp | grep 8080输出包含PID, 然后kill -9 PID释放端口。

17. grep、sed、awk分别有什么用途?

grep: 文本搜索过滤。根据正则表达式匹配行并输出。常用-i忽略大小写, -v反向匹配, -r递归目录, -n显示行号。例如: grep "error" /var/log/syslog。
sed: 流编辑器。用于文本替换、删除、插入。例如: sed 's/old/new/g' file 全局替换; sed -i 's/old/new/g' file直接修改文件。
awk: 文本处理语言, 适合处理结构化数据 (如CSV、日志)。按字段分割, 支持条件、循环、格式化输出。例如: awk '{print 1,3}' file打印第1、3列; awk -F',' '{sum+=2} END{print sum}' 计算第二列和。三者常配合管道使用: ps aux | grep nginx | awk '{print 2}' 获取nginx的PID。

18. 如何查看文件末尾的实时更新?

使用tail -f filename命令。-f (follow) 会持续输出文件新增的内容, 常用于监控日志文件。按Ctrl+C终止。另外, tail -F在文件被删除后重新创建时仍能跟踪 (处理日志轮转)。其他工具: less +F filename, 进入less后按F进入实时跟踪模式, 按Ctrl+C暂停。

19. 如何查找文件?

find: 最强大, 按各种条件查找。find / -name "filename" 从根目录按名称查

找（可能很慢）。`find /home -type f -size +10M` 查找大于10M的文件。`find /var -mtime -7` 最近7天内修改的文件。`locate`：基于预先构建的数据库快速搜索，速度极快。使用前需`updatedb`更新数据库。`locate filename`。
`whereis`：查找命令的二进制、源码和man页。`whereis ls`。`which`：查找命令的绝对路径（从PATH中查找）。`which nginx`。`grep -r`：在文件内容中搜索文本：`grep -r "pattern" /path`。

20. Linux文件权限中的rwx分别代表什么？数字分别是什么？

`r`：读权限，对应数字4。对于文件，可以读取内容；对于目录，可以列出目录内容（需要`ls`）。`w`：写权限，对应数字2。对于文件，可以修改内容；对于目录，可以在目录中创建、删除、重命名文件（需要`x`配合）。`x`：执行权限，对应数字1。对于文件，可以运行；对于目录，可以进入目录（`cd`）以及访问目录下文件的元信息。数字组合：将rwx对应数字相加。 $7(4+2+1)=rwx$ ， $6(4+2)=rw-$ ， $5(4+1)=r-x$ ， $4=r--$ ， $3(2+1)=-wx$ ， $2=-w-$ ， $1=--x$ ， $0=---$ 。
权限示例：`-rw-r--r--` 表示文件类型（-），属主读写（6），属组只读（4），其他只读（4），数字为644。

4月8日 八股补充(网络,数据库,Linux各15道)

计算机网络

1. TCP和UDP的适用场景分别是什么？如何选择？

TCP适用于对可靠性要求高、可以容忍一定延迟的场景，如HTTP/HTTPS网页访问、文件传输（FTP）、邮件（SMTP/POP3）、远程登录（SSH/Telnet）等。UDP适用于对实时性要求高、可以容忍少量丢包的场景，如实时音视频通话（RTP）、直播、在线游戏、DNS域名解析、SNMP网络管理等。面试时可补充：如果需要兼顾可靠性和实时性，可以在应用层基于UDP实现可靠传输（如QUIC、KCP、RUDP）。

2. 什么是QUIC协议？它相比TCP有什么优势？

QUIC（Quick UDP Internet Connections）是基于UDP的传输协议，由Google提出，是HTTP/3的底层协议。核心优势：① 0-RTT/1-RTT快速握手，减少连接建立延迟；② 连接迁移，通过连接ID代替四元组，切换网络（如WiFi切4G）时连接不中断；③ 多路复用无队头阻塞，一个流丢包不影响其他

流；④ 内置前向纠错，可恢复部分丢包而无需重传。面试时可以说：QUIC解决的是TCP在移动网络和高延迟场景下的痛点，已在Chrome、Cloudflare等大规模落地。

3. 什么是WebSocket？和HTTP有什么区别？

WebSocket是支持全双工通信的应用层协议。区别：HTTP是半双工，客户端请求才能收到响应，服务器无法主动推送；WebSocket建立连接后，双方可随时主动发送消息。实现方式：WebSocket通过HTTP Upgrade机制完成握手，从HTTP协议升级为WebSocket协议，之后在同一TCP连接上全双工通信。适用场景：实时聊天、在线游戏、股票行情推送等。

4. 什么是同源策略？跨域有哪些解决方案？

同源策略是浏览器的安全机制，要求两个页面的协议、域名、端口完全相同才允许共享资源（如Cookie、DOM访问、Ajax请求）。跨域解决方案：① CORS（跨域资源共享），服务器设置Access-Control-Allow-Origin等响应头；② JSONP，仅限GET请求；③ 反向代理，将前端请求转发到同源后端；④ WebSocket，不受同源策略限制；⑤ postMessage跨窗口通信；⑥ Nginx代理转发；⑦ Node中间件；⑧ iframe + window.name / document.domain。大厂面试重点考察CORS原理，需能讲清预检请求（OPTIONS）的触发条件。

5. CORS预检请求是什么？什么时候触发？

CORS预检请求（Preflight Request）是指浏览器在发送“非简单请求”前，先发送一个OPTIONS请求询问服务器是否允许跨域。触发条件：请求方法不是GET/HEAD/POST，或POST的Content-Type不是 application/x-www-form-urlencoded / multipart/form-data / text/plain，或请求头中包含自定义字段（如Authorization）。预检请求通过后，浏览器才会发送实际请求。

6. HTTP/1.0、HTTP/1.1、HTTP/2.0、HTTP/3.0的核心区别是什么？

① HTTP/1.0：短连接，每次请求需重新建立TCP连接。② HTTP/1.1：默认持久连接（Keep-Alive）、管道化（Pipelining，有队头阻塞）、Host头支持、断点续传。③ HTTP/2.0：二进制分帧、多路复用（解决队头阻塞）、头部压缩（HPACK）、服务器推送。④ HTTP/3.0：基于QUIC协议，彻底解决TCP层的队头阻塞，支持0-RTT快速握手和连接迁移。

7. 什么是DNS？DNS解析过程是怎样的？

DNS（Domain Name System）负责将域名解析为IP地址。解析过程：① 浏览器检查DNS缓存；② 检查操作系统hosts文件；③ 向本地DNS服务器发起

递归查询；④ 本地DNS服务器从根域名服务器开始迭代查询：根域名返回顶级域（如.com）地址 → 顶级域名返回权威域名服务器地址 → 权威域名服务器返回最终IP；⑤ 返回给客户端并缓存。补充：DNS默认使用UDP 53端口，响应超过512字节时切换为TCP。

8. 什么是ARP协议？ARP欺骗是什么？

ARP（Address Resolution Protocol）用于将IP地址解析为MAC地址，在局域网内实现二层通信。工作原理：广播ARP请求“谁有IP X？”目标主机单播回复自己的MAC地址，发送方将映射存入ARP缓存。ARP欺骗（ARP Spoofing）：攻击者伪造ARP响应，将自身MAC地址与目标IP关联，使网络流量被劫持。防御措施：静态ARP绑定、ARP防火墙、使用交换机端口安全等。

9. ICMP协议是什么？ping和traceroute的工作原理？

ICMP（Internet Control Message Protocol）用于网络层的错误报告和诊断。ping使用ICMP Echo Request/Reply报文，测试网络连通性和RTT。traceroute利用IP报文的TTL字段：第一次发送TTL=1的报文，第一个路由器丢弃并返回ICMP超时报文（获取第一跳地址）；然后TTL=2、3...直到到达目标主机，通过分析返回的ICMP报文逐跳追踪路由路径。

10. 什么是TIME_WAIT？为什么需要它？为什么是2MSL？

TIME_WAIT是TCP四次挥手中主动关闭方在收到对端FIN后的状态，持续2MSL（Maximum Segment Lifetime，通常2分钟）。作用：① 确保最后一个ACK能被对端收到，如果ACK丢失，对端会重发FIN，TIME_WAIT状态下的连接可以重发ACK；② 让网络中的旧报文段自然消失，防止影响后续新建连接的通信。2MSL的时间保证旧报文段在网络中完全消散。

11. TCP的粘包问题是什么？如何解决？

粘包指TCP传输中多个数据包被合并成一个，或一个数据包被拆分成多个，因为TCP是面向字节流的协议，不保留应用层消息边界。解决方案：① 固定消息长度；② 在消息头中声明消息体长度（最常见）；③ 使用特殊分隔符（如换行符）；④ 应用层协议设计时明确边界（如HTTP用空行分隔头部和Body）。面试时强调：粘包是TCP字节流特性导致的，不是协议Bug，UDP不会粘包。

12. 什么是RTO和RTT？TCP如何计算超时重传时间？

RTT（Round-Trip Time）是报文从发送到收到确认的时间。RTO（Retransmission TimeOut）是超时重传的时间阈值。TCP采用Jacobson算法

动态计算RTO：维护平滑RTT（SRTT）和RTT偏差（RTTVAR），公式为 $RTO = SRTT + 4 \times RTTVAR$ 。早期版本使用固定值，现代TCP根据网络状况自适应调整，避免因RTT波动导致的误判或不必要的重传。

13. 什么是NAT？它的工作原理是什么？

NAT（Network Address Translation）将私有IP地址映射为公网IP地址，解决IPv4地址短缺问题。工作原理：当内网主机访问外网时，NAT设备将数据包的源IP和端口（如192.168.1.2:12345）替换为公网IP和新的端口（如203.0.113.1:56789），并记录映射关系；收到响应时反向还原。常见类型：静态NAT（一对一定映射）、动态NAT（池中取用）、NAPT（端口级复用，最常用）。

14. 什么是隧道技术？VPN如何工作？

隧道技术将一种协议的数据包封装在另一种协议中传输，常见于VPN、IPv6 over IPv4等场景。VPN工作原理：① 客户端与VPN服务器建立加密隧道（如IPsec、OpenVPN、WireGuard）；② 客户端发送的内网数据包被封装并加密后发送给服务器；③ 服务器解密解封装后转发到目标网络；④ 响应数据再经隧道返回。VPN实现原理是隧道+加密，实现跨公网的安全通信。

15. 如何排查网络故障？常用的网络诊断工具有哪些？

① ping 测试连通性和延迟；② traceroute / tracert 追踪路由路径；③ netstat / ss 查看端口监听和连接状态；④ tcpdump / Wireshark 抓包分析；⑤ telnet / nc 测试端口连通性；⑥ nslookup / dig 诊断DNS；⑦ curl 测试HTTP接口；⑧ iperf 测试带宽吞吐量。排查思路：从应用层往下，先确认服务状态 → 检查端口 → 检查防火墙 → 检查路由可达性 → 抓包分析。

数据库

1. MySQL有哪些索引类型？

① **主键索引**：唯一非空，InnoDB中为聚簇索引。② **唯一索引**：索引列值必须唯一。③ **普通索引**：无唯一性约束。④ **全文索引**：支持大文本的快速检索（InnoDB从5.6开始支持）。⑤ **组合索引**：多个列组成的索引，遵循最左前缀原则。⑥ **空间索引**：用于GIS地理数据。

2. 什么情况下索引会失效？

① 对索引列使用函数或表达式（`WHERE YEAR(date) = 2020`）；② 隐式类型转换（`varchar`列传整数）；③ `LIKE`以通配符开头（`'%abc'`）；④ `OR`条件中有非索引列；⑤ 联合索引不满足最左前缀；⑥ 查询结果超过表30%时MySQL可能全表扫描；⑦ 使用 `!=`、`<>` 或 `IS NULL / IS NOT NULL`（视`NULL`值比例而定）。

3. 如何分析一条SQL的执行效率？

使用 `EXPLAIN` 命令查看执行计划，关键字段：`type`（访问类型，`const/ref/range/index/ALL`从好到差）、`key`（实际使用的索引）、`rows`（预估扫描行数）、`Extra`（`Using index`表示覆盖索引，`Using filesort`表示需额外排序，`Using temporary`表示需临时表）。慢查询定位：开启慢查询日志（`slow_query_log=ON`，设置 `long_query_time`），用 `pt-query-digest` 分析。

4. MySQL的三大日志分别是什么？

① **binlog**：服务层日志，记录所有写操作，用于主从复制和基于时间点的数据库恢复。② **redo log**：InnoDB引擎日志，记录数据页的物理修改，保证事务持久性（崩溃恢复时重做已提交事务），是WAL（Write-Ahead Logging）机制的核心。③ **undo log**：InnoDB引擎日志，记录操作前的数据，用于事务回滚和MVCC多版本读取。

5. SQL优化的常用方法有哪些？

① 避免 `SELECT *`，只查询需要的字段；② 合理使用索引，避免索引失效；③ 使用 `EXPLAIN` 分析执行计划；④ 优化JOIN：小表驱动大表，确保关联字段有索引；⑤ 避免子查询，尽量改写成JOIN；⑥ 分批处理大范围操作（如 `DELETE` 分批删除）；⑦ 使用覆盖索引减少回表；⑧ 合理设计分页，避免大 `offset`（可改用游标或延迟关联）；⑨ 数据归档，将历史数据迁移到归档表。

6. MySQL中有哪几种锁？

按粒度分：表级锁（`MyISAM`）和行级锁（`InnoDB`）。按类型分：① 共享锁（`S`锁，`SELECT ... LOCK IN SHARE MODE`）允许其他事务读；② 排他锁（`X`锁，`SELECT ... FOR UPDATE`）阻止其他读写。按算法分（`InnoDB`行锁）：`Record Lock`（记录锁）、`Gap Lock`（间隙锁，防止幻读）、`Next-Key Lock`（记录锁+间隙锁）。按意向分：意向共享锁/意向排他锁，用于表级和行级锁的协同。

7. 什么是间隙锁？如何解决幻读？

间隙锁（Gap Lock）锁住索引记录之间的间隙，不锁具体记录，阻止其他事务在间隙中插入新数据。InnoDB在可重复读隔离级别下，通过 **MVCC + 间隙锁** 组合解决幻读：快照读使用MVCC读取历史版本，当前读（`SELECT ... FOR UPDATE`）使用间隙锁防止新数据插入。例如 `SELECT * FROM user WHERE age BETWEEN 10 AND 20 FOR UPDATE` 会锁住该范围内的间隙。

8. 什么是死锁？如何避免和解决？

死锁是两个或多个事务相互持有对方需要的锁，形成循环等待。四个必要条件：互斥、请求与保持、不可剥夺、循环等待。避免方法：① 按相同顺序访问资源；② 事务尽量短小；③ 使用 `SELECT ... FOR UPDATE` 时注意索引；④ 适当降低隔离级别。解决：MySQL自动检测死锁，选择回滚代价较小的事务（`innodb_deadlock_detect` 控制）。

9. 什么是覆盖索引？为什么能优化查询性能？

覆盖索引指一个索引包含了查询所需的所有字段，无需回表到聚簇索引获取行数据。例如索引 `(name, age)`，查询 `SELECT name, age FROM user WHERE name = '张三'`，所有数据都在索引中，直接返回。优势：减少磁盘I/O次数，因为二级索引的叶子节点比聚簇索引小，一次I/O能加载更多索引页。

10. 主从复制的延迟如何产生？如何解决？

延迟原因：从库SQL线程是单线程回放binlog（5.6版本前），主库高并发写入时从库回放跟不上。解决方案：① MySQL 5.7+启用多线程并行复制（`slave_parallel_workers`，支持基于库或基于逻辑时钟）；② 主库写入时尽量使用单线程或分散写入压力；③ 对延迟不敏感的业务可读从库；④ 使用半同步复制，确保至少一个从库同步；⑤ 采用组复制（MGR）或Galera Cluster等强一致性方案。

11. 什么是数据库死锁？如何处理？

见第8题。

12. 如何定位和解决MySQL慢查询？

见第3题。

13. 数据库的三大范式是什么？为什么要反范式化？

三大范式：① 1NF：列不可再分；② 2NF：非主键列完全依赖主键（消除部分依赖）；③ 3NF：非主键列不传递依赖主键。反范式化是故意增加冗余字段

以减少JOIN次数，以空间换时间。例如订单表冗余用户姓名，避免每次查询都JOIN用户表。适用场景：读多写少、对响应时间要求高的业务。

14. 什么是分库分表？水平分表和垂直分表的区别？

分库分表是将数据分散到多个数据库或表中，解决单库单表数据量过大问题。**垂直分表**：按字段拆分宽表，将热点字段和冷字段分开。**垂直分库**：按业务模块拆分到不同数据库服务器。**水平分表**：将同一张表的数据按规则（hash、range）分散到多张表。**水平分库**：数据分散到多个数据库实例。挑战：跨库JOIN、分布式事务、全局主键、分页排序等需特殊处理，常用中间件有ShardingSphere、MyCat、Vitess。

15. Redis缓存穿透、缓存雪崩、缓存击穿的区别和解决方案？

穿透：查询不存在的数据，绕过缓存直达DB → 布隆过滤器或缓存空对象。
雪崩：大量缓存同时过期，请求直冲DB → 过期时间加随机值、热点数据永不过期、Redis高可用。
击穿：热点key过期瞬间高并发 → 互斥锁（SETNX）只让一个线程重建缓存、永不过期+后台异步更新。

Linux

1. 进程和线程的区别是什么？

① **资源**：进程拥有独立的地址空间和系统资源，线程共享进程的地址空间和资源（如打开的文件）。② **开销**：进程创建和切换开销大（需要分配PCB、页表等），线程开销小。③ **通信**：进程间通信需要IPC机制（管道、消息队列、共享内存等），线程间可直接读写共享内存。④ **健壮性**：一个线程崩溃可能导致整个进程崩溃，进程间相互隔离。⑤ **调度**：线程是CPU调度的基本单位，进程是资源分配的基本单位。

2. 进程间通信（IPC）的方式有哪些？

① **管道**：匿名管道（|，父子进程）和命名管道（FIFO，无亲缘关系进程）。② **信号**：异步事件通知（如SIGKILL、SIGTERM）。③ **消息队列**：消息链表，按类型读取。④ **共享内存**：最快的IPC，多进程映射同一块物理内存，需配合信号量同步。⑤ **信号量**：控制对共享资源的并发访问。⑥ **Socket**：跨主机或本机进程通信（Unix Domain Socket效率高）。⑦ **内存映射文件（mmap）**：将文件映射到进程地址空间。

3. 用户态和内核态的区别？为什么需要区分？

用户态和内核态是CPU的两个特权级别。用户态：受限访问硬件和内核内存，只能执行非特权指令。内核态：可执行所有指令，访问所有内存和硬件资源。区别的目的在于保护系统安全，防止应用程序错误导致系统崩溃。用户态程序需通过**系统调用**（如open、read、write）切换到内核态才能访问资源（如磁盘、网络）。切换方式：系统调用、异常、外设中断。

4. 什么是虚拟内存？分页和分段的区别？

虚拟内存是操作系统内存管理技术，为每个进程提供独立的、连续的虚拟地址空间，实际映射到不连续的物理内存。好处：进程隔离、内存利用率高、支持超大地址空间（超过物理内存）。

- **分页**：将虚拟地址和物理地址划分为固定大小的页（如4KB），通过页表映射，消除外部碎片。
- **分段**：按逻辑段（代码段、数据段等）划分，段大小可变，易产生外部碎片。
- 现代OS通常采用**分页+分段**结合（段页式）。

5. 页面置换算法有哪些？

当物理内存不足时，OS需将某些页面换出到磁盘。常见算法：① **FIFO**（先进先出）：简单但可能换出常用页；② **LRU**（最近最少使用）：理论最优但硬件实现成本高；③ **Clock/NRU**（时钟算法/最近未使用）：近似LRU，Linux采用改进版；④ **LFU**（最不常用）：统计访问频率；⑤ **OPT**（最优置换）：理论上的最佳但不可实现。

6. 什么是死锁？产生死锁的必要条件？

死锁是指多个进程相互等待对方持有的资源，导致都无法继续执行。四个必要条件：① **互斥**：资源一次只能被一个进程占用；② **请求与保持**：进程持有资源的同时请求新资源；③ **不可剥夺**：已分配的资源不能被强制剥夺；④ **循环等待**：多个进程形成资源循环等待链。预防和避免方法：破坏任一必要条件（如资源有序分配法、银行家算法）。

7. 什么是僵尸进程？什么是孤儿进程？

僵尸进程：子进程已终止，但父进程未调用 `wait()` / `waitpid()` 回收其PCB（进程控制块），导致进程表中残留“僵尸”条目。会占用进程表资源。**孤儿进程**：父进程已终止，子进程仍在运行，被init进程（PID=1）收养，由init负责回收。孤儿进程不会造成资源泄漏，反而是一种防止僵尸的机制。解决方案：父进程需正确回收子进程，或使用双fork技巧让init收养。

8. fork()函数执行过程是怎样的？

fork() 通过系统调用创建新进程（子进程）。① 内核为子进程分配新的PCB和PID；② 复制父进程的地址空间（写时复制技术，COW，实际只复制页表）；③ 子进程和父进程的代码段相同，执行位置相同。区别： fork() 在父进程中返回子进程的PID，在子进程中返回0。写时复制：父子进程共享物理内存页，只有当一方尝试写入时才真正复制，节省内存和时间。

9. 什么是epoll？它和select/poll的区别是什么？

epoll是Linux的高性能I/O多路复用机制。与select/poll区别：

- **select**：监控文件描述符有上限（默认1024），每次调用需遍历全量集合，效率随fd数线性下降。
- **poll**：无上限，但仍需遍历，随fd数增长性能下降。
- **epoll**：红黑树存储关注fd，就绪队列存储活跃fd，只需遍历就绪队列。支持**边缘触发**（ET）和**水平触发**（LT）。epoll是高性能服务器（如Nginx、Redis）的核心技术。

10. top命令输出的各个指标含义？

- **load average**：1/5/15分钟平均负载（正在运行+等待运行的进程数），理想值不超过CPU核心数。
- **%us**：用户态CPU使用率； **%sy**：内核态； **%id**：空闲； **%wa**：I/O等待。
- **RES**：进程实际使用的物理内存； **VIRT**：虚拟内存总量； **SHR**：共享内存。
- **交互操作**：按P按CPU排序，按M按内存排序，按H显示线程。

11. 如何查看内存使用情况？

① `free -h`：总内存、已用、空闲、buff/cache、可用内存。② `cat /proc/meminfo`：更详细的内存信息。③ `top / htop` 中按M按内存排序。④ `vmstat 1`：实时监控内存、swap、IO。⑤ `ps aux --sort=-%mem | head -10`：查看内存占用前10的进程。

12. 如何查看日志？有哪些常用命令？

① `tail -f`：实时追踪日志末尾，常用于监控。② `head -n 100`：查看文件前100行。③ `grep "ERROR" app.log`：过滤关键字。④ `less`：分页查看，支持搜索（/）和前后翻页。⑤ `cat`：适合小文件一次性输出。⑥

`journalctl -u nginx.service`：查看systemd服务日志（Linux发行版常用）。

13. 如何查看端口占用和网络状态？

① `netstat -tunlp | grep :80`：-t TCP，-u UDP，-n不解析域名，-l监听状态，-p显示进程。② `ss -tunlp | grep :80`：netstat的升级版，更快。③ `lsof -i:80`：直接查看端口占用进程。④ `ss -s`：查看网络统计摘要。

14. 什么是软链接和硬链接？区别是什么？

硬链接（`ln source target`）：多个文件名指向同一inode，删除一个不影响其他；不能跨文件系统，不能链接目录。**软链接/符号链接**（`ln -s source target`）：类似快捷方式，存储源文件路径；可跨文件系统，可链接目录；删除源文件后软链接失效（悬空链接）。查看方式：`ls -li` 查看inode号，硬链接inode相同，软链接文件类型为l。

15. 如何杀死进程？kill命令的信号有什么区别？

`kill PID`：发送SIGTERM（15），允许进程清理后退出（优雅终止）。`kill -9 PID`：发送SIGKILL，强制立即终止，进程无法捕获，可能导致数据丢失。
`kill -l`：列出所有信号。其他命令：`pkill -f nginx` 按进程名杀，`killall nginx` 杀所有匹配进程。常用信号：SIGINT（2，Ctrl+C中断）、SIGTERM（15，正常终止）、SIGKILL（9，强制终止）、SIGHUP（1，终端关闭或配置重加载）。
